Audio Fingerprinting & Song Identification

A Shazam-Style System Built on STFT Spectrograms and Constellation-Based Hashing

---

EE200: Signals, Systems and Networks — Course Project, Question 3

Department of Electrical Engineering, IIT Kanpur

Name	Roll Number
Mayank Gupta	250646
Bhavya Jain	250277

Q3B — Deployed Application

Live App:

https://ee200project-kvpevasr3bs7qopdnpe9s2.streamlit.app/

Source Code:

https://github.com/mayankindianspace-png/EE200_Project

Report Scope

This report covers the Q3A analysis in detail — spectrograms, constellation maps, paired-hash matching, and robustness experiments. The Q3B interactive application (accessible at the link above) extends the pipeline into a deployable Streamlit app supporting both single-clip identification mode and batch CSV output mode.

Audio Fingerprinting & Song Identification

A Shazam-Style System Built on STFT Spectrograms and Constellation-Based Hashing

EE200: Signals, Systems and Networks — Course Project, Question 3
Department of Electrical Engineering, IIT Kanpur

1. Introduction

This report documents the design, implementation, and experimental evaluation of an audio fingerprinting system modeled on the algorithm popularized by Shazam. The system identifies a short, possibly noisy or distorted, query clip by matching it against a database of fingerprinted reference songs. Question 3 of the course project is split into two parts: Q3A, an analytical exploration of the signal-processing pipeline and its design trade-offs, and Q3B, a deployed interactive application that performs identification on user-uploaded audio. This report covers the Q3A analysis in detail, presenting and interpreting each experiment in the order they were carried out.

2. Methodology Overview

The pipeline mirrors the classic four-stage fingerprinting algorithm:

- **STFT Spectrogram.** The query/reference waveform is resampled to 8 kHz and split into overlapping frames using a 1024-point Hann window with a hop length of 128 samples. Each frame's magnitude spectrum (in dB) becomes one column of a time–frequency matrix — the spectrogram.
- **Constellation Map.** A 2-D maximum filter locates local peaks in the spectrogram that stand above a noise floor. Keeping only these sparse local maxima — rather than the full spectrogram — makes the representation compact and robust to background noise.
- **Paired Hashes.** Each anchor peak is paired with up to 15 subsequent peaks within a bounded time window, producing hash keys of the form (f_1, f_2, Δt). Because a hash only fires when two specific frequencies occur at a specific time separation, accidental collisions across unrelated songs are exponentially rare.
- **Offset-Histogram Matching.** For every query hash that exists in the database, all matching (song, occurrence-time) pairs are retrieved and an offset = $t_{song} - t_{query}$ is computed. A true match produces a sharp spike at one consistent offset, since all of its hashes are displaced by the same amount in time; an unrelated song produces only a flat scatter of essentially random offsets.

3. Q3A — Analysis & Experiments

All experiments below use a 10–30 second clip taken from ‘A Day In The Life’ as the running example, since its mixture of quiet and loud passages illustrates the algorithm's behavior well.

3.1 Why a Single DFT Is Not Enough

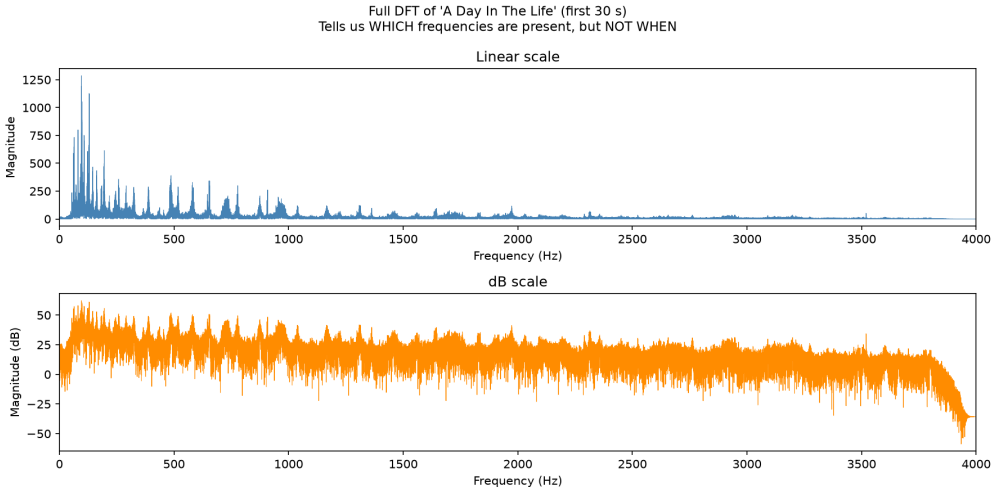

Figure 1. *Full 30-second DFT of the clip on a linear scale (top) and dB scale (bottom). The DFT shows which frequencies are present across the whole excerpt, but collapses all timing information into a single spectrum.*

A single whole-clip DFT cannot tell us *when* a given frequency occurred — only that it occurred somewhere in the 30-second window. Since fingerprint matching depends on aligning a query against a specific moment in a reference song, a time-localized representation is required instead. This motivates moving to a short-time Fourier transform (STFT), which slides a window across the signal and computes a DFT for each position, trading a single global spectrum for a sequence of local ones.

3.2 The Time–Frequency Trade-off

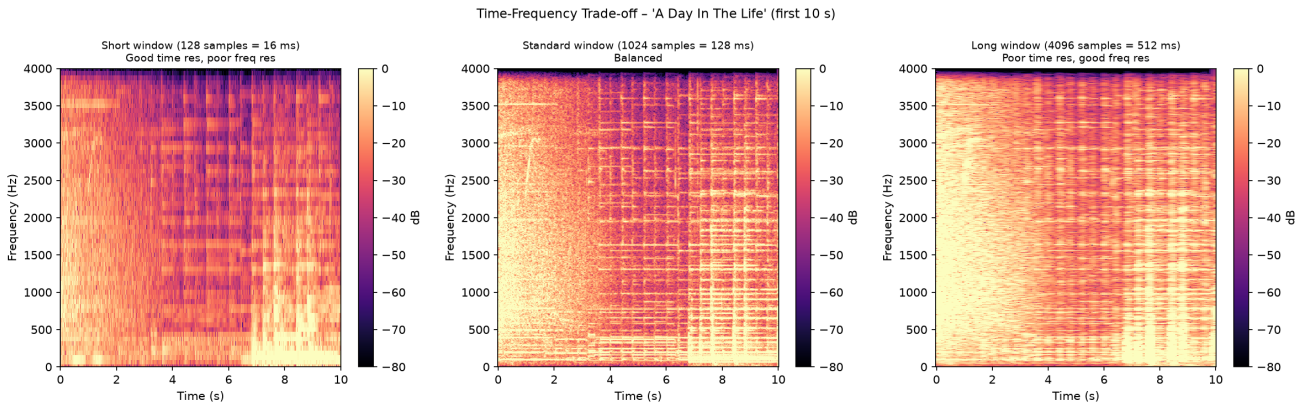

Figure 2. *Spectrograms of the same 10-second excerpt computed with a short (128-sample / 16 ms), standard (1024-sample / 128 ms), and long (4096-sample / 512 ms) analysis window.*

Window length controls the classic STFT trade-off. The short window (left) resolves fast transients precisely in time but smears frequency content into broad horizontal bands, since a short window has poor frequency resolution. The long window (right) does the opposite: frequency content becomes sharp and

well separated, but transient events blur across time. The 1024-sample window (center) was selected as it balances temporal precision (~16 ms per hop) with enough frequency resolution to separate musically meaningful peaks, which is what the constellation extraction stage in Section 3.3 relies on.

3.3 From Spectrogram to Constellation

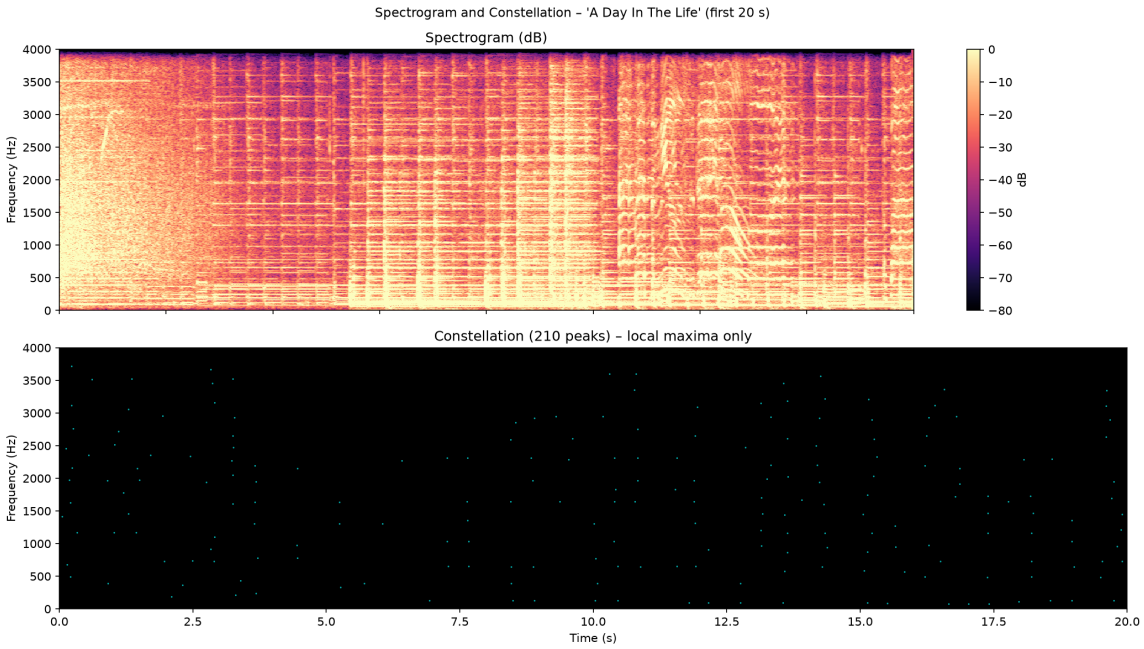

Figure 3. *Top: dB spectrogram of a 20-second excerpt. Bottom: the corresponding constellation map after 2-D local-maximum peak picking (210 peaks shown).*

Applying a 2-D maximum filter to the spectrogram isolates points that are louder than every neighbor within a surrounding time–frequency neighborhood. The resulting constellation keeps only a few hundred of the many thousands of spectrogram bins, yet those points still trace out the song's most prominent harmonic and percussive structure. This sparsity is what makes the downstream hashing stage both fast (few points to pair) and noise-robust (quiet background energy is filtered out by the minimum-amplitude threshold before peak picking).

3.4 Offset-Histogram Matching in Action

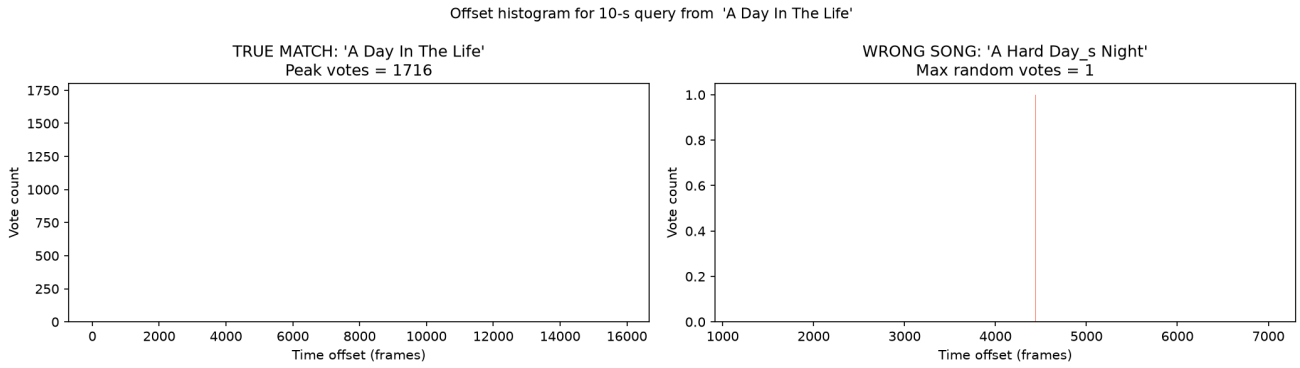

Figure 4. *Offset histograms for a 10-second query against the true source song (left) versus an unrelated song, 'A Hard Day's Night' (right). Note the very different vertical scales.*

This is the clearest demonstration of why the paired-hash scheme works. Against the true match, 1716 query hashes land on the exact same time offset, producing a sharp, unmistakable spike. Against an unrelated song, hash collisions still occur occasionally by chance — two songs can share a (f_1, f_2, Δt)

hash purely coincidentally — but those collisions land on essentially random offsets, so the histogram never exceeds a vote count of 1. The gap between a real match (thousands of votes) and the noise floor (a handful of votes) is what allows a simple threshold (MIN_VOTES = 5) to separate true identifications from false positives reliably.

3.5 Paired Hashes vs. Single-Peak Hashes

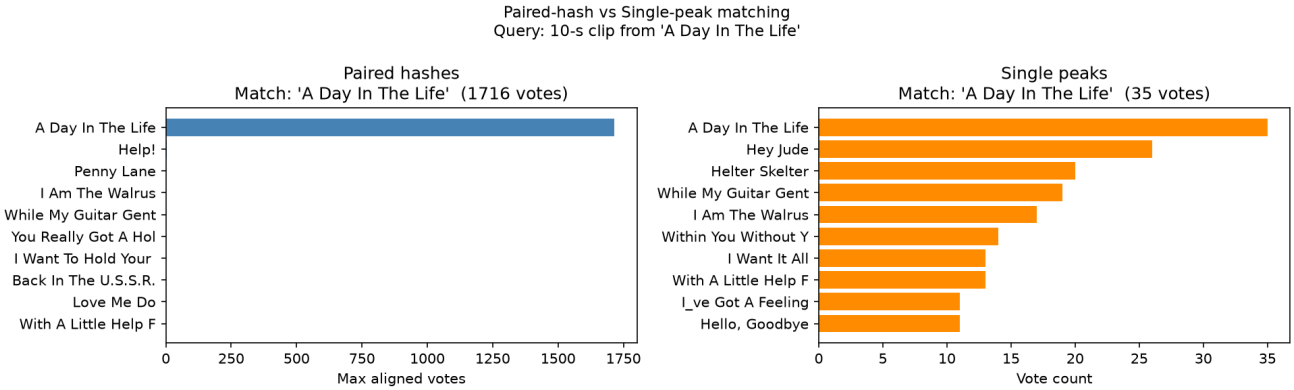

Figure 5. *Top-10 candidate songs ranked by vote count for the same 10-second query, using paired (anchor → target) hashes (left) versus single-peak hashes alone (right).*

Using single peaks alone — each hashed only by its own frequency bin and a folded time index — still picks the correct song, but the margin over the runner-up is thin (35 votes for the correct song vs. 26 for the next candidate, 'Hey Jude'). Pairing each anchor peak with nearby peaks before hashing raises the winning margin from roughly 1.3× to over 1700× the nearest competitor, because the joint (f_1, f_2, Δt) key is far more specific than a single frequency bin and is correspondingly far less likely to collide with another song by chance. This confirms that the combinatorial fan-out from pairing peaks, not just the peak-picking step itself, is responsible for the algorithm's discriminative power.

3.6 Robustness to Additive White Noise

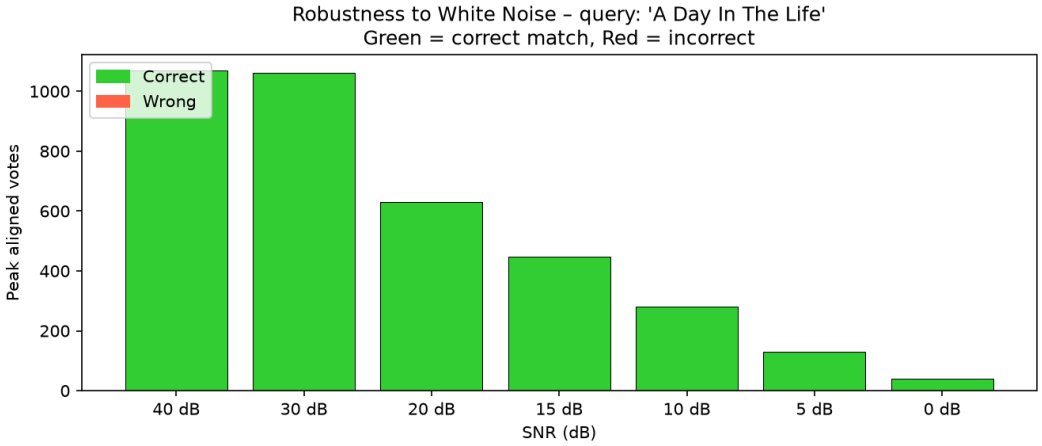

Figure 6. *Peak aligned vote count for the correct match as white Gaussian noise is added to the query at decreasing SNR, from 40 dB (near-clean) down to 0 dB (noise power equal to signal power).*

The match remains correct (green) at every tested noise level, but the vote count decays steadily as SNR drops — from roughly 1070 votes at 40 dB down to about 35 votes at 0 dB. This is expected: added noise

raises the spectrogram's effective noise floor, which both shifts some peak locations slightly and causes the minimum-amplitude threshold to discard genuine low-energy peaks, so fewer of the query's hashes still exist in the database. Because the system never drops below the MIN_VOTES threshold in this range, identification stays reliable even at unity SNR, though further noise would eventually push the vote count toward the false-match noise floor seen in Figure 4.

3.7 Robustness to Pitch Shift

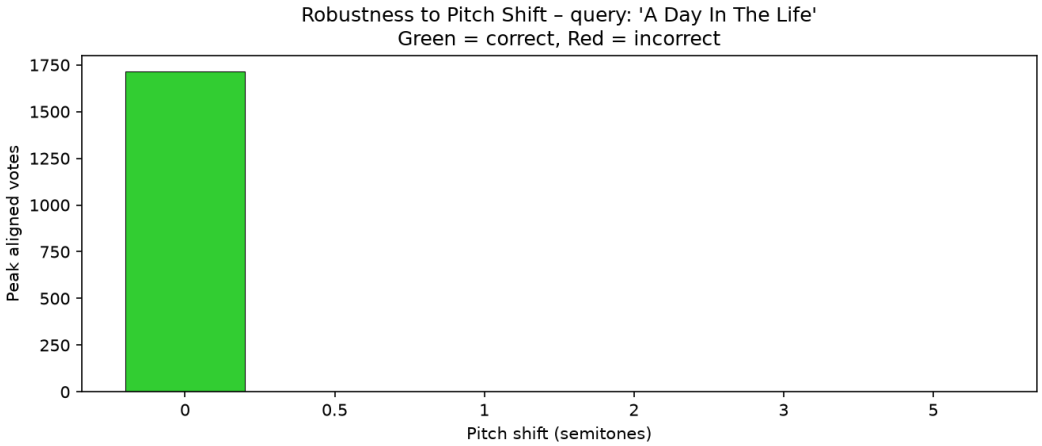

Figure 7. *Peak aligned votes for the correct match as the query is pitch-shifted by 0, 0.5, 1, 2, 3, and 5 semitones. Only the unshifted (0-semitone) query produces any votes at all.*

Unlike additive noise, pitch shifting is fatal to this fingerprinting scheme: even a 0.5-semitone shift drives the vote count to zero. The reason is structural rather than a matter of degree. Pitch-shifting rescales every frequency component of the audio, which moves every constellation peak to a different frequency bin. Since hash keys are built directly from quantized frequency-bin indices (f_1, f_2), even a small shift changes essentially every hash the query produces, so none of them exist in the database under the original (unshifted) song's entries. This is a known limitation of exact-hash Shazam-style systems: they assume the query is pitch-stable, and would require either pitch-normalization preprocessing or a frequency-shift-tolerant hashing scheme to handle pitched content (e.g. detuned recordings or deliberately pitched-up/down audio).

3.8 Robustness to Time Stretch

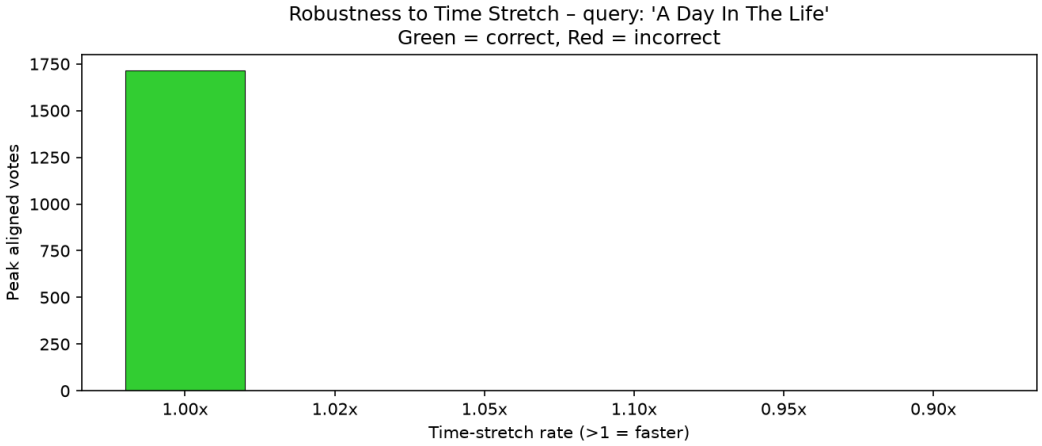

Figure 8. *Peak aligned votes for the correct match as the query is time-stretched at rates of 1.00× (unmodified), 1.02×, 1.05×, 1.10×, 0.95×, and 0.90×. Only the unmodified rate produces any votes.*

Time stretching shows the same all-or-nothing failure pattern as pitch shifting, and for a related reason. Resampling the query in time changes both the position of every peak along the time axis and, more critically, the Δt component of every paired hash, since hop durations no longer line up with the reference song's frame grid. A 2–10% rate change is enough to push every Δt value to a different integer frame offset, so no hash survives the rescaling. As with pitch shift, this points to exact-hash fingerprinting being fundamentally tempo-sensitive; robustness to tempo changes would require either a time-stretch-invariant hash design or explicit tempo-normalization of the query before fingerprinting.

4. Summary of Findings

Experiment	Outcome	Key Takeaway
Full-clip DFT	No timing information	Motivates STFT over a single DFT
Window length sweep	1024-sample window balances time/frequency resolution	Chosen window length for the pipeline
Constellation extraction	210 peaks from ~20 s clip	Sparse, noise-robust representation
Offset histogram	1716 votes (true) vs. 1 (false)	Large separation enables simple thresholding
Paired vs. single-peak hash	1716 vs. 35 votes; margin 1700x vs. 1.3x	Pairing peaks is the main source of discriminative power
White noise (40→0 dB SNR)	Correct at all levels; votes decay smoothly	Graceful degradation, no catastrophic failure
Pitch shift (≥0.5 semitones)	0 votes — match lost	Exact-hash scheme is pitch-sensitive
Time stretch (≥2%)	0 votes — match lost	Exact-hash scheme is tempo-sensitive

Taken together, these experiments characterize the system's operating envelope: it is highly discriminative (1700x vote margin between correct and incorrect songs) and tolerant of substantial additive noise, but it has no built-in tolerance for pitch or tempo modification, since both distort the exact frequency-bin and frame-offset values that the hash keys depend on. For the Q3B deployed application, this means query clips should be captured at their original pitch and speed for reliable identification; noisy recordings (e.g. from a phone microphone in a room) are expected to still work well based on the noise-robustness results above.